

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Murshidha.M	Reg. No.:	
Section / Batch:	A	Date:	04/09/2026

Code of Conduct

I Murshidha.M (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Murshidha

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

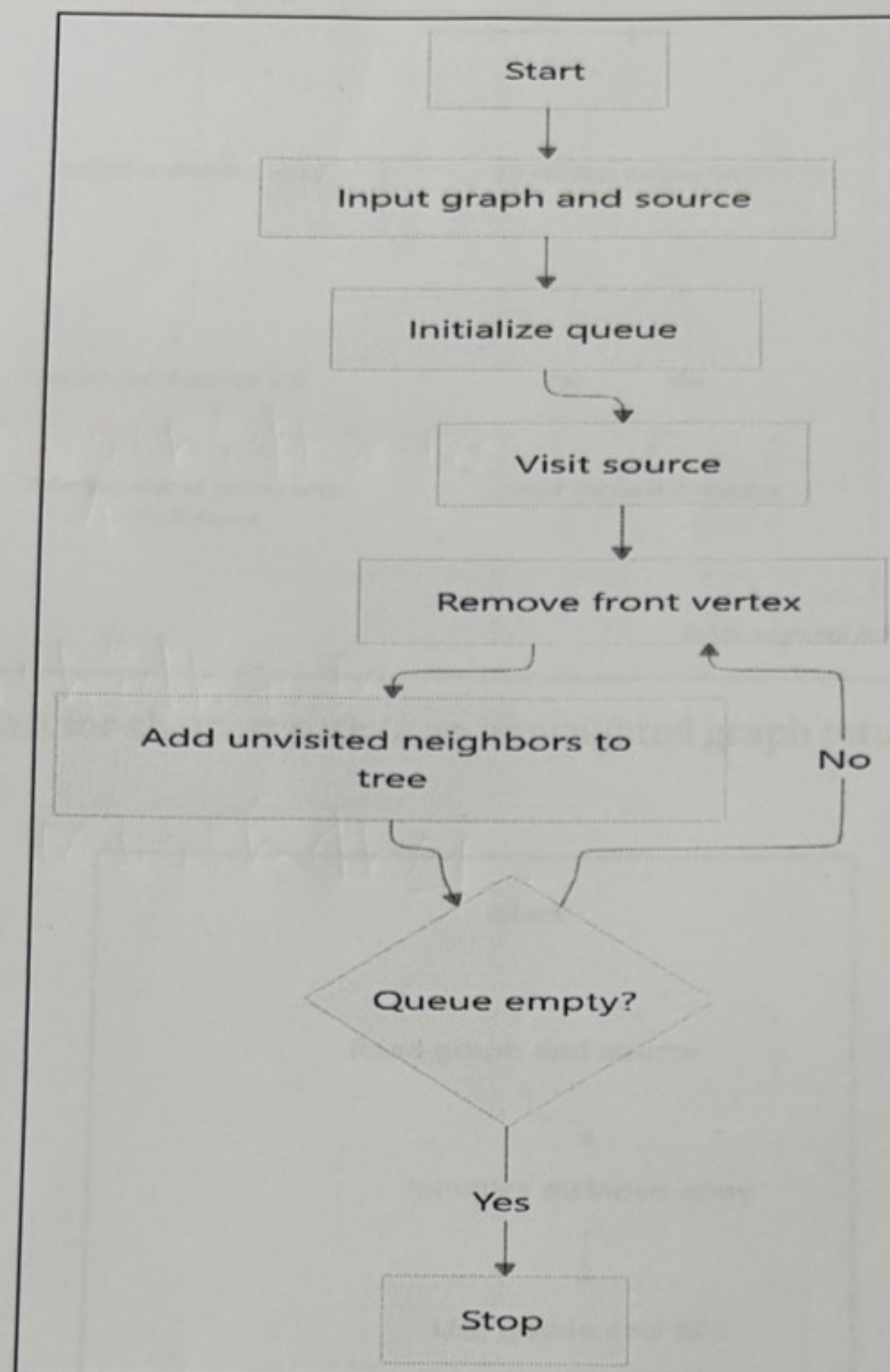
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

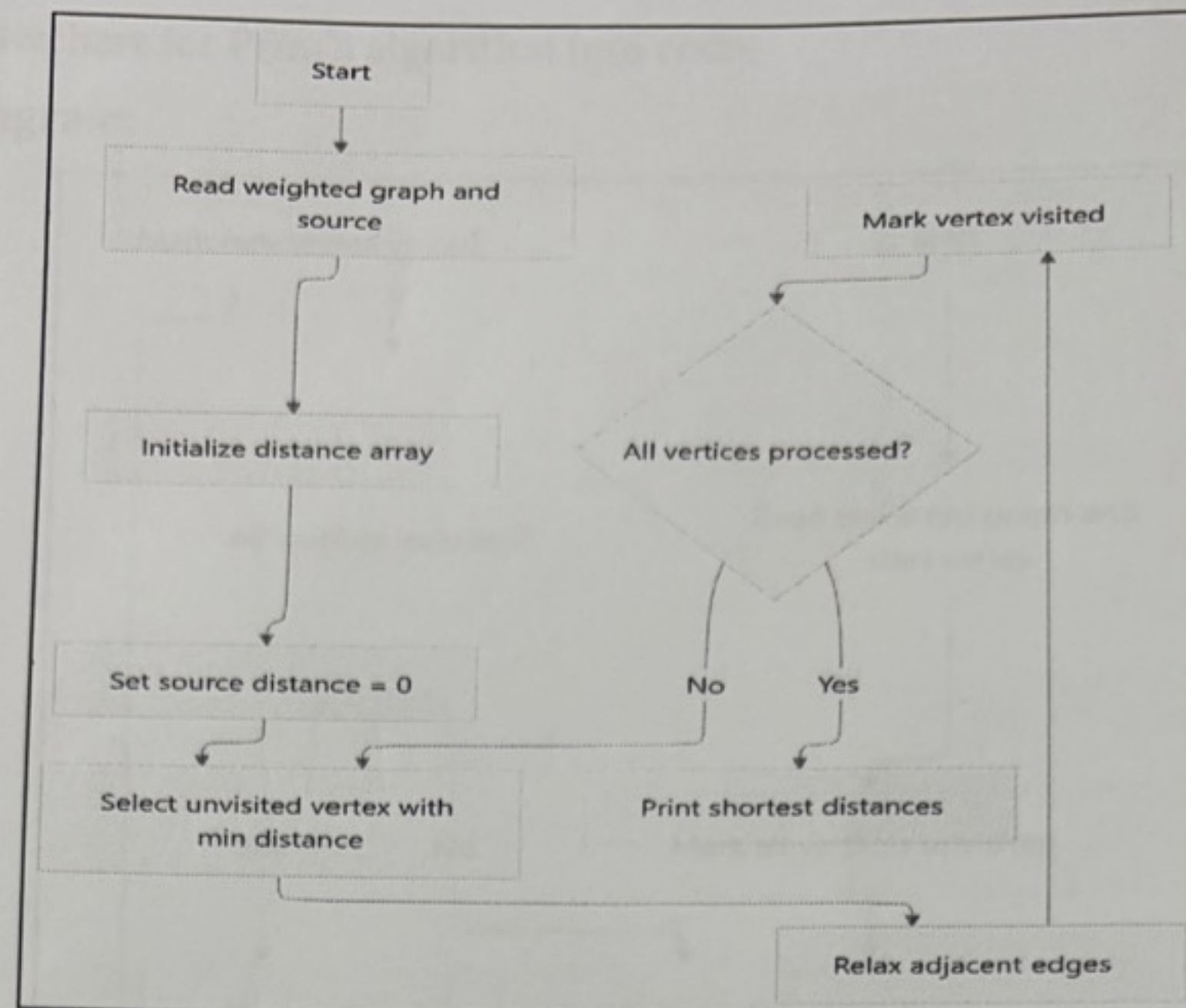
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

(20 Marks)

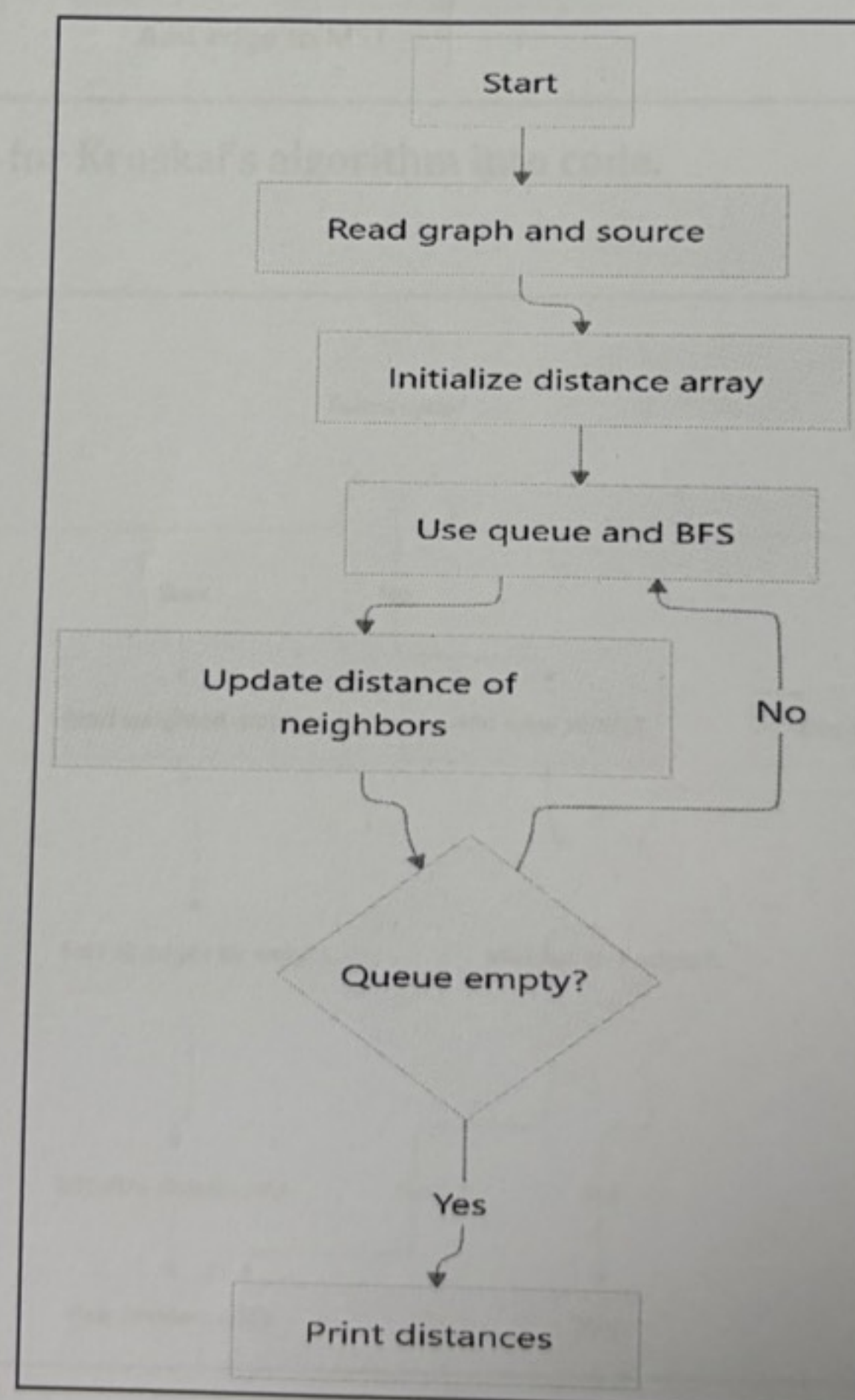
Flowchart Diagram:



3. Convert the flowchart for shortest path in an unweighted graph into code.

(20 Marks)

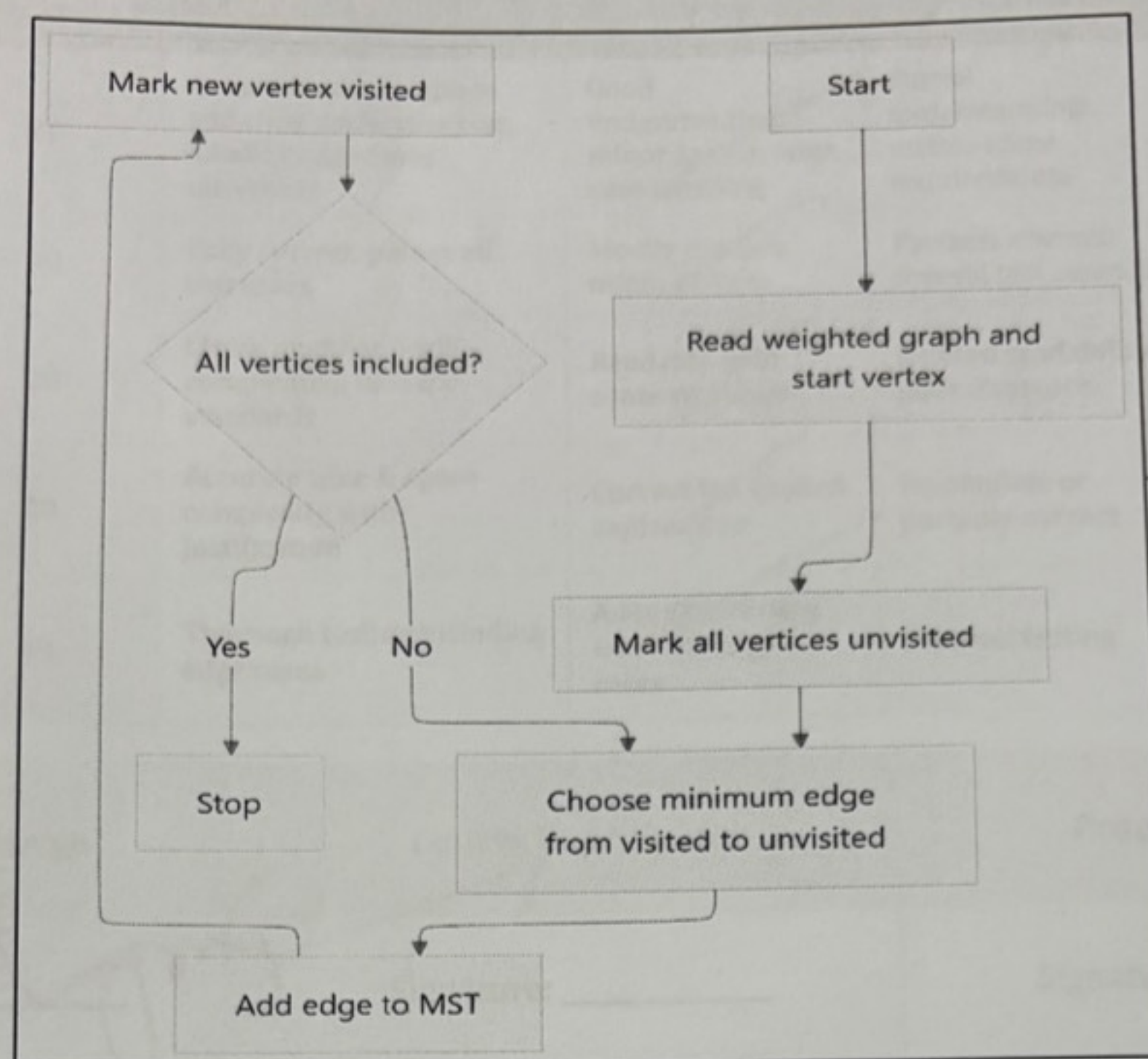
Flowchart Diagram:



4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

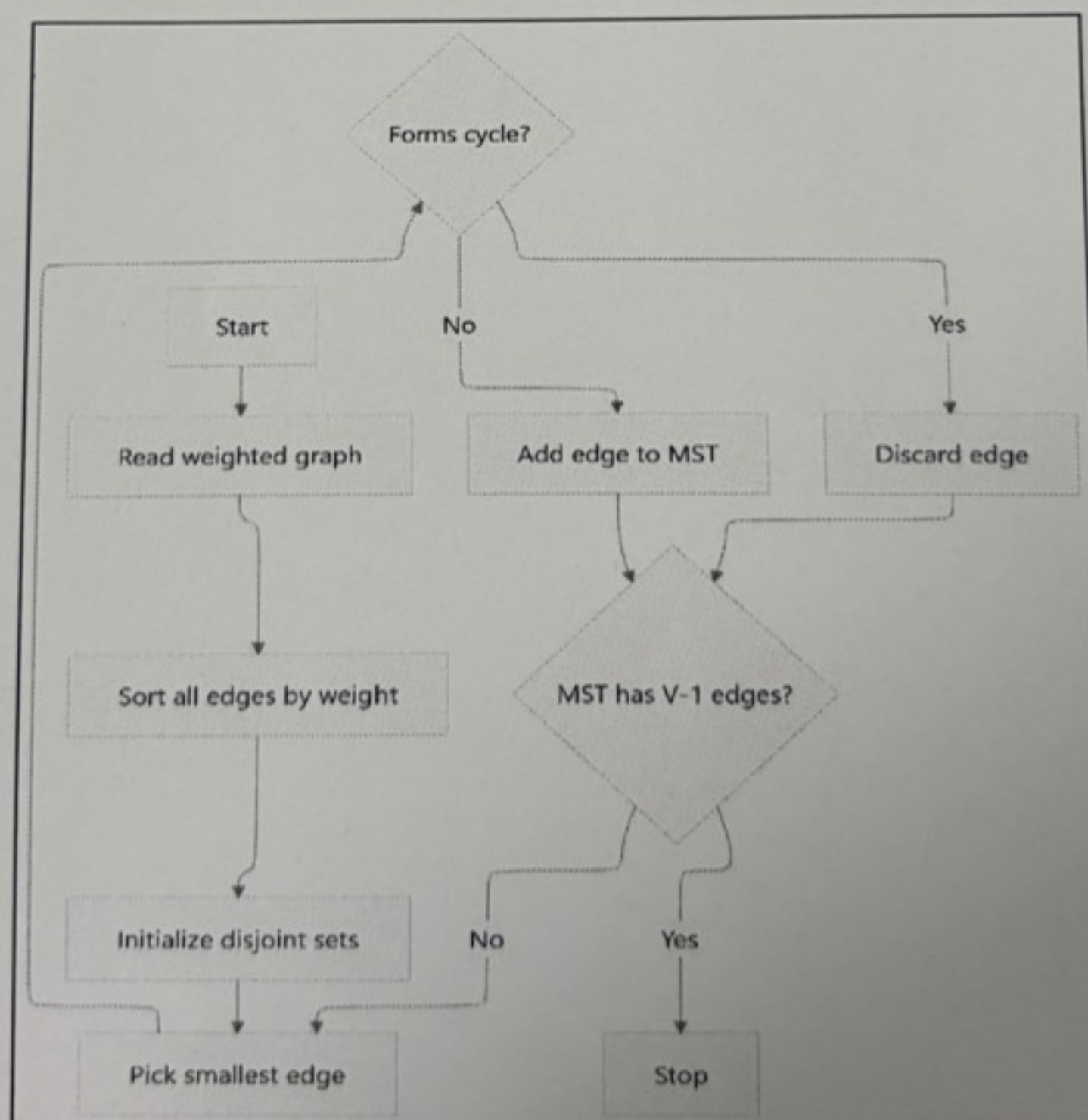
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

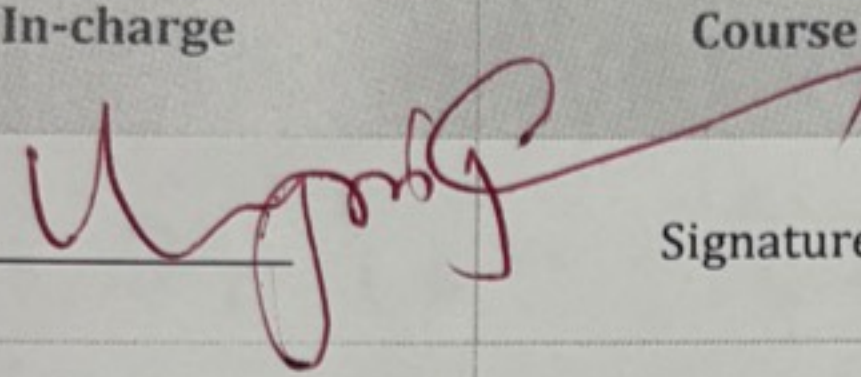
(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

BFS - Tree

Simple code:

```
#include <stdio.h>

int main() {
    int graph[10][10], visited[10] = {0};
    int queue[10], front = 0, rear = 0;
    int n, start, i, v;

    printf("Enter number of vertices:");
    scanf("%d", &n);

    printf("Enter adjacency matrix: \n");
    for (i = 0; i < n; i++)
        for (v = 0; v < n; v++)
            scanf("%d", &graph[i][v]);

    printf("Enter starting vertex:");
    scanf("%d", &start);
    visited[start] = 1;
    queue[rear++] = start;
    printf("BFS: ");
    while (front < rear) {
        v = queue[front++];
        printf("%d ", v);
        for (i = 0; i < n; i++) {
            if (graph[v][i] == 1 &&
```



```

visited[i] == 0){
    visited[i] = 1;
    queue[rear++] = i;
}
}
}
return 0;
}

```

complexity: time - $O(V^2)$, space - $O(V)$

2. Dijkstra's shortest path simple code.

```

#include <stdio.h>
#define INF 9999
int main() {
    int cost[10][10], dist[10], visited[10] = {0};
    int n, start, i, j, min, u;
    printf("Enter number of vertices:");
    scanf("%d", &n);
    printf("Enter cost matrix: \n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    printf("Enter source vertex:");

```



```
printf ("%d → %d\n", i, dist[i]);  
return 0;
```

}

complexity: time - $O(V^2)$, space - $O(V)$

3. Shortest in unweighted graph.

```
#include <stdio.h>
```

```
int main () {
```

```
    int graph[10][10], dist[10], visited[10] = {0};
```

```
    int queue[10], front = 0, rear = 0;
```

```
    int h, start, i, v;
```

```
    printf ("Enter number of vertices: ");
```

```
    scanf ("%d", &n);
```

```
    printf ("Enter adjacency matrix: \n");
```

```
    for (i = 0; i < n; i++)
```

```
        for (v = 0; v < n; v++)
```

```
            scanf ("%d", &graph[i][v]);
```

```
    printf ("Enter source vertex: ");
```

```
    scanf ("%d", &start);
```

```
    for (i = 0; i < n; i++)
```

```
        for (i = 0; i < n; i++)
```

```
            for (v = 0; v < n; v++)
```

```
                scanf ("%d", &start);
```

```
    for (i = 0; i < n; i++)
```

```
        for (v = 0; v < n; v++)
```

```
            scanf ("%d", &graph[i][v]);
```



```
scanf ("%d", &start);
```

```
for (i=0; i<n; i++).
```

```
dist [i] = cost [start][i];
```

```
dist [start] = 0;
```

```
visited [start] = 1;
```

```
for (i=0; i<n; i++) {
```

```
    min = INF;
```

```
    u = -1;
```

```
    for (j=0; j<n; j++) {
```

```
        if (!visited [j] & dist [j] < min) {
```

```
            min = dist [j];
```

```
            u = j;
```

```
        }
```

```
    }
```

```
    if (u == -1)
```

```
        break;
```

```
    visited [u] = 1;
```

```
    for (i=0; i<n; i++) {
```

```
        if (!visited [i] & dist [u] + cost [u][i]  
            < dist [i])
```

```
            dist [i] = dist [u] + cost [u][i];
```

```
    }
```

```
}
```

```
printf ("Shortest distances: \n");
```

```
for (i=0; i<n; i++)
```



```
printf("Enter source vertex:");  
scanf("%d", &start);
```

```
printf("Shortest distances:\n");  
for (i=0; i<n; i++)  
    printf("%d → %d\n", i, dist[i]);  
return 0;
```

complexity: time - $O(V^2)$, space - $O(V)$

4. Prim's Algorithm - Minimum Spanning Tree.

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main()
```

```
{  
    int graph[10][10], selected[10] = {0};
```

```
    printf("Enter cost matrix:\n");
```

```
    for (i=0; i<n; i++)
```

```
        printf("Edges in MST:\n");
```

```
        while (edges < n-1){
```

```
            min = INF;
```



```

if (!Selected [j] & & graph [i][j] < min)
{
    min = graph [i][j];
    x = i;
    y = j;
}
}
}

```

edges ++;

```

printf ("minimum cost = %d", cost);
return 0;
}

```

complexity: time - $O(V^2)$, space - $O(V)$

5. Kruskal's Algorithm - MST

```
#include <stdio.h>
```

```
struct edge {
```

```
    int u, v, w;
```

```
};
```

```
int parent [20];
```

```
int find (int x) {
```

```
    while (parent [x] != x)
```

```
        x = parent [x];
```


return x;

```
void unite (int a, int b) {  
    parent [ find (a) ] = find (b);  
}  
int main () {
```

```
    Struct Edge [50], temp;  
    printf ("Enter vertices & edges:");  
    scanf ("%d %d", &n, &m);  
    printf ("Enter edges (u v weight): \n");
```

```
    v e [i].w);  
    for (i = 0; i < n; i++)
```

```
        parent [i] = i;  
        if (e [i].w > e [i+1].w) {  
            temp = e [i];
```

```
            e [i] = e [i+1];
```

```
            e [i+1] = temp;
```

```
            cost += e [i].w;
```

```
            unite (e [i].u, e [i].v);
```

```
            count ++;
```

Ujjwal